

TECHNICAL DOCUMENTATION

Architecture & Implementation Reference

This document provides a comprehensive technical reference for the Taskflow application, covering its software architecture, layering, request handling, authentication mechanism, and database design. All content is derived directly from review of the source code contained in this repository.

[Angular 21 \(standalone + signals\)](#)[Spring Boot 3.4.4 / Java 21](#)[PostgreSQL 16](#)[JWT \(jjwt 0.12.6\)](#)

Document Control

Field	Detail
Document Title	Taskflow — Architecture & Implementation Reference
Version	1.0
Status	Final
Date Prepared	15 July 2026
Source Repository	task-management-system/
Scope	Backend (backend/src/main/java/com/taskmanager), Frontend (frontend/src/app), Database (database/schema.sql)

Contents

1. Project Overview	
2. Technology Stack	
3. Software Architecture	
4. Folder & Package Structure	
5. Major Classes & Responsibilities	
6. How the Frontend, Backend, and Database Communicate	
7. Authentication Flow (JWT)	
8. Database Structure	
9. API Request Flow	
10. API Reference	

Every statement in this document is traceable to a specific file within the codebase (backend Java sources under `backend/src/main/java/com/taskmanager` , frontend TypeScript under `frontend/src/app` , and `database/schema.sql`). Where the implementation exhibits a limitation or an unused code path, this is identified explicitly within a dedicated callout rather than omitted.

SECTION 1

Project Overview

Taskflow is a full-stack, multi-user task management system. Each user registers an account, signs in, and manages a personal list of tasks, including creating, editing, searching, filtering, sorting, and deleting them. Every task belongs to exactly one user, and the system enforces that each user may view or modify only their own tasks.

The system is built as two independently deployable applications that communicate over HTTP:

- A **backend REST API** (Spring Boot, Java 21) that owns all business logic, security, and persistence, listening on `localhost:8080`.
- A **frontend single-page application** (Angular 21) that renders the UI and talks to the API exclusively over JSON, served in development on `localhost:4200`.

Both applications sit in front of a **PostgreSQL 16** database, which the backend accesses through Hibernate/JPA; the frontend does not communicate with the database directly under any circumstance.

Core Capabilities (Confirmed by Source Review)

Authentication

Users register and sign in with an email and password. Passwords are hashed with BCrypt (`SecurityConfig.passwordEncoder()`). Sessions are stateless: every request is re-authenticated by way of a JWT bearer token (`JwtService`, `JwtAuthenticationFilter`).

Task CRUD

Full create, read, update, and delete operations on tasks, scoped per owner at the repository layer (`TaskRepository.findByIdAndUserId`), such that ownership cannot be bypassed by supplying another user's task identifier.

Search, Filter, Sort, and Paginate

Keyword search over title and description, filtering by status and priority, sorting by a whitelisted set of fields, and pagination — all composed dynamically using JPA Specifications (`TaskSpecifications`) within `TaskService.search()`.

Consistent Error Handling

Every exception type — validation, not-found, conflict, invalid credentials, access-denied, or unexpected — is routed through a single `@RestControllerAdvice` (`GlobalExceptionHandler`) into one consistent JSON structure (`ApiError`).

Deployment Model: Taskflow is a decoupled client-server system rather than a monolithic application with server-rendered pages. No HTML templating exists on the backend (no Thymeleaf or JSP); the Spring Boot application is a pure JSON API, and the entire user interface resides within the Angular application.

SECTION 2

Technology Stack

Backend — backend/pom.xml

Component	Technology	Role
Language / runtime	Java 21	Language level pinned in <code><java.version></code>
Framework	Spring Boot 3.4.4 (parent BOM)	Application framework, auto-configuration, embedded Tomcat
Web	spring-boot-starter-web	REST controllers, embedded servlet container
Persistence	spring-boot-starter-data-jpa (Hibernate)	ORM, repositories, Specification-based queries
Validation	spring-boot-starter-validation	Bean Validation annotations on request DTOs
Security	spring-boot-starter-security	Filter chain, password encoding, authentication manager
JWT	io.jsonwebtoken (jjwt) 0.12.6 — api / impl / jackson	Signing and parsing JSON Web Tokens
Database driver	org.postgresql:postgresql (runtime scope)	JDBC driver for PostgreSQL
Build	Maven + spring-boot-maven-plugin	Build/packaging, <code>mvn spring-boot:run</code>
Testing	spring-boot-starter-test, spring-security-test	Declared as dependencies (no test sources currently present in <code>src/test</code>)

Frontend — frontend/package.json

Component	Technology	Role
Framework	Angular ^21.0.0	Standalone components (no <code>NgModule</code> s anywhere in the app)
Language	TypeScript 5.9.2	Strict-mode application code
Reactivity	Angular Signals (<code>signal</code> , <code>computed</code>)	Local component/service state — no NgRx or global store library is used
HTTP	RxJS 7.8 + <code>HttpClient</code>	Observable-based API calls with functional interceptors
Routing	@angular/router	Lazy-loaded standalone routes with functional guards
Build	Angular CLI / @angular-devkit/build-angular	<code>ng serve</code> / <code>ng build</code>

Database

Component	Detail
Engine	PostgreSQL 16 (per README prerequisites)
Schema management	<code>spring.jpa.hibernate.ddl-auto: update</code> — Hibernate creates/updates tables from the <code>@Entity</code> classes on every boot. <code>database/schema.sql</code> is a hand-maintained equivalent DDL script for manual/production provisioning.

Tables `users` , `tasks` (see Section 8)

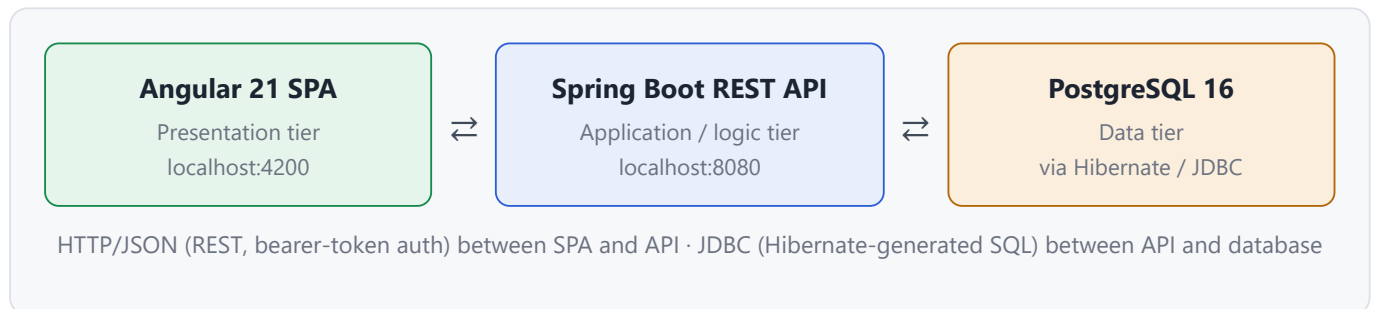
Configuration — `application.yml`

All environment-specific values are externalized with sensible dev defaults and can be overridden by environment variables at runtime:

Setting	Env var	Default
Server port	<code>SERVER_PORT</code>	8080
Datasource URL / user / password	<code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_NAME</code> , <code>DB_USERNAME</code> , <code>DB_PASSWORD</code>	localhost / 5432 / task_manager / postgres / postgres
JWT signing secret	<code>APP_JWT_SECRET</code>	Base64 dev secret embedded in <code>application.yml</code> (must be overridden in production)
JWT expiration	<code>APP_JWT_EXPIRATION_MS</code>	86400000 ms (24 hours)
Allowed CORS origin(s)	<code>APP_CORS_ORIGINS</code>	http://localhost:4200

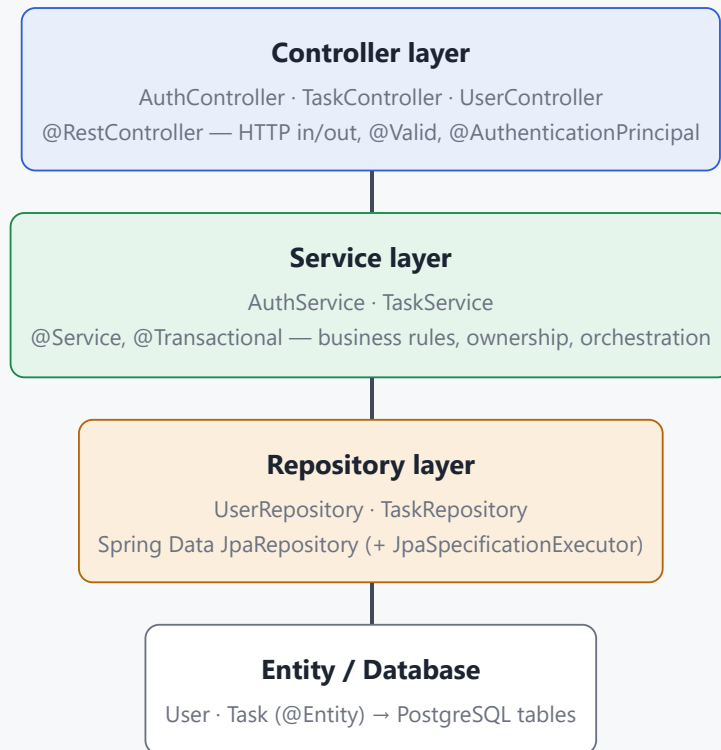
Software Architecture

Taskflow is a **three-tier client-server system**, and the backend tier is internally organized as a classic **Layered Architecture**, following the Controller → Service → Repository → Entity structure around which Spring Data applications are conventionally built. It is **not** an MVC application in the traditional server-rendered sense, and it does not employ Hexagonal or Clean Architecture module boundaries (there are no separate `domain`, `application`, or `infrastructure` modules, nor explicit port interfaces distinct from the Spring Data repositories).



Rationale: Layered Architecture, Not MVC

- **No server-side view layer exists.** There are no Thymeleaf or JSP templates anywhere within `backend/src`; every controller returns a `ResponseEntity<DTO>` serialized directly to JSON. The "view" responsibility belongs entirely to the separately deployed Angular application, and therefore describing the backend as MVC would be inaccurate.
- **Strict one-directional dependency flow.** Every controller (`AuthController`, `TaskController`, `UserController`) depends only on a service; every service (`AuthService`, `TaskService`) depends only on a repository; every repository depends only on a JPA entity. No reverse dependency exists — a repository never calls a service, and a service never accesses `HttpServletRequest` — which is the defining characteristic of a layered architecture.
- **Clearly delineated responsibility per layer**, evidenced directly within the code: controllers contain no business logic (they validate input via `@Valid` and translate outcomes into HTTP status codes); services contain all business rules (ownership checks, uniqueness checks, sort-field whitelisting); repositories contain no logic beyond declarative Spring Data query derivation and `Specification` composition.



Backend layering. Two cross-cutting concerns wrap every request outside this vertical stack: the Spring Security filter chain (runs before the controller layer) and GlobalExceptionHandler (intercepts anything thrown by any layer).

Package Organization: Feature-Based, Layered by Class

At the top level, backend code is grouped **by feature or domain** (`auth`, `task`, `user`, `security`, `config`, `exception`) rather than by technical role. Within each feature package, however, classes remain cleanly separated by role: for example, `task/` contains `TaskController`, `TaskService`, `TaskRepository`, and a nested `dto/` package side by side. This represents a deliberate and common hybrid convention, keeping each feature's controller, service, repository, and DTO classes discoverable together while preserving the layered dependency direction described above.

Cross-cutting layers

Security filter chain

`SecurityConfig` registers `JwtAuthenticationFilter` ahead of Spring Security's standard `UsernamePasswordAuthenticationFilter`, with `SessionCreationPolicy.STATELESS` (no `HttpSession` is ever created). This intercepts every request before it reaches a controller.

Centralized error translation

`GlobalExceptionHandler` (`@RestControllerAdvice`) converts every exception type raised anywhere — validation, not-found, conflict, bad credentials, access-denied, or unhandled — into one consistent `ApiError` JSON shape with the correct HTTP status.

DTO boundary

Request/response records (`TaskRequest` / `TaskResponse`, `UserResponse`, `AuthResponse`, etc.) decouple the public API

Auditing

`@EnableJpaAuditing` on `TaskManagerApplication`, combined with `@CreatedDate` / `@LastModifiedDate` on `Task` and

contract from the JPA entities — e.g. `UserResponse` deliberately omits the password field that `User` carries.

`@CreateDate` on `User`, auto-stamps timestamps without any explicit service code.

Frontend Architecture

The Angular application follows the framework's recommended **feature-based folder convention**, implemented with modern standalone APIs rather than classic `NgModule`s:

- `core/` — cross-cutting concerns shared across the application: `models` (TypeScript types mirroring backend DTOs), `services` (HTTP clients), `guards` (route protection), and `interceptors` (cross-cutting HTTP behavior).
- `features/` — page-level, domain-specific components: `auth` (login, register) and `dashboard` (task list and task form).
- `shared/` — reusable presentational components used across features, currently limited to `navbar`.

Application state is managed with **Angular Signals** (`signal()`, `computed()`) local to each service or component; no NgRx, Akita, or other global state management library is used. Route guards (`authGuard`, `guestGuard`) and HTTP interceptors (`authInterceptor`, `errorInterceptor`) are implemented as plain functions using Angular's functional guard and interceptor APIs, rather than as classes.

Folder & Package Structure

Backend — backend/src/main/java/com/taskmanager

```

com.taskmanager
├─ TaskManagerApplication.java  Spring Boot entry point (@EnableJpaAuditing)
├─ auth/                        registration & login
│   ├── AuthController.java
│   ├── AuthService.java
│   └─ dto/ LoginRequest · RegisterRequest · AuthResponse · UserResponse
├─ user/                        user identity & profile
│   ├── User.java              @Entity implementing Spring Security UserDetails
│   ├── Role.java              enum: ROLE_USER, ROLE_ADMIN
│   ├── UserRepository.java
│   └─ UserController.java     GET /api/users/me
├─ task/                        task domain (largest package)
│   ├── Task.java              @Entity, ManyToOne owner
│   ├── Priority.java · TaskStatus.java  enums
│   ├── TaskRepository.java    JpaRepository + JpaSpecificationExecutor
│   ├── TaskSpecifications.java  dynamic query building blocks
│   ├── TaskService.java
│   ├── TaskController.java
│   └─ dto/ TaskRequest · TaskResponse · PageResponse<T>
├─ security/                    JWT + Spring Security plumbing
│   ├── JwtService.java
│   ├── JwtAuthenticationFilter.java
│   ├── JwtAuthenticationEntryPoint.java
│   └─ AppUserDetailsService.java
├─ config/
│   └─ SecurityConfig.java     filter chain, CORS, password encoder, auth beans
└─ exception/                  cross-cutting error handling
    ├── GlobalExceptionHandler.java
    ├── ApiError.java
    ├── ResourceNotFoundException.java
    └─ EmailAlreadyExistsException.java

```

Frontend — frontend/src/app

```

app/
├─ app.component.ts            root shell — shows <app-navbar> when authenticated
├─ app.config.ts               providers: router, HttpClient + interceptors
├─ app.routes.ts               route table, lazy-loaded standalone components
├─ core/
│   ├── models/ task.model.ts · user.model.ts
│   ├── services/ auth.service.ts · task.service.ts
│   ├── guards/ auth.guard.ts  authGuard, guestGuard
│   └─ interceptors/ auth.interceptor.ts · error.interceptor.ts
├─ features/
│   ├── auth/
│   │   ├── login/ login.component.ts
│   │   └─ register/ register.component.ts
│   └─ dashboard/

```

```
|      |— dashboard.component.ts  search/filter/sort/paginate, task grid
|      |— task-form.component.ts  create/edit modal
|— shared/navbar/ navbar.component.ts
```

Repository root

```
task-management-system/
|— backend/      Spring Boot Maven project
|— frontend/    Angular CLI project
|— database/    schema.sql  reference DDL (tables are also auto-created by Hibernate)
|— docs/        existing design notes (00-07)
|— README.md
```

Major Classes & Responsibilities

Bootstrap

TaskManagerApplication `com.taskmanager`

The Spring Boot entry point. `@SpringBootApplication` triggers component scanning and auto-configuration; `@EnableJpaAuditing` activates the `@CreatedDate` / `@LastModifiedDate` annotations used on `User` and `Task`.

Package: `config`

SecurityConfig

Defines the entire security posture in one place: builds the `SecurityFilterChain` (CORS enabled, CSRF disabled — safe here because the API is stateless and uses bearer tokens rather than cookies), whitelists `/api/auth/**` and `/actuator/health` as public, requires authentication for everything else, sets `SessionCreationPolicy.STATELESS`, wires the custom `JwtAuthenticationEntryPoint` for unauthenticated responses, and inserts `JwtAuthenticationFilter` before Spring's default `UsernamePasswordAuthenticationFilter`. Also exposes the `PasswordEncoder` (BCrypt), `AuthenticationProvider` (`DaoAuthenticationProvider`), and `AuthenticationManager` beans used by `AuthService`.

Package: `security`

JwtService

Issues and verifies JWTs using the jjwt library. The signing key is derived from `app.jwt.secret` (HMAC-SHA); tokens carry the user's **email** as the subject, an `issuedAt`, and an `expiration` computed from `app.jwt.expiration-ms` (24h by default). `isTokenValid()` checks both subject match and non-expiry.

JwtAuthenticationFilter

A `OncePerRequestFilter` that runs on every request. It reads the `Authorization` header and, if it is a `Bearer` token, extracts the email, loads the corresponding `UserDetails`, and, if the token is valid, populates a `UsernamePasswordAuthenticationToken` into `SecurityContextHolder`. Any exception raised during parsing is caught and the security context is cleared accordingly, deferring the 401 response to the entry point rather than throwing from within the filter.

JwtAuthenticationEntryPoint

Fires whenever Spring Security rejects an unauthenticated request to a protected endpoint. Writes a JSON `ApiError` body (status 401) directly to the response, so unauthenticated failures have the exact same shape as every other API error.

AppUserDetailsService

The Spring Security `UserDetailsService` implementation. It looks up a user **by email** via `UserRepository.findByEmail`; the email address serves as the "username" for authentication purposes throughout this system.

Package: `auth`

AuthController

`POST /api/auth/register` and `POST /api/auth/login`. Both endpoints are unauthenticated (permitted in `SecurityConfig`). The controller delegates directly to `AuthService` without additional logic, returning `201 Created` for registration and `200 OK` for login.

AuthService

`register()`: checks `existsByEmail`, throws `EmailAlreadyExistsException` if taken, otherwise BCrypt-encodes the password, builds a `User` defaulted to `ROLE_USER`, persists it, and issues a JWT. `login()`: delegates credential verification entirely to Spring Security's `AuthenticationManager` (which throws `BadCredentialsException` on failure), then issues a fresh JWT for the authenticated principal. Both methods return an `AuthResponse`.

DTOs — LoginRequest, RegisterRequest, AuthResponse, UserResponse

Java records with Bean Validation constraints (`@NotBlank`, `@Email`, `@Size`). `UserResponse.from(User)` and `AuthResponse.of(token, user)` are static factories that project the entity into a client-safe shape — notably, `UserResponse` never carries the password hash.

Package: `user`

User `@Entity` implements `UserDetails`

Dual-purpose class: it is both the JPA entity mapped to the `users` table **and** the Spring Security principal. Its `getUsername()` returns the email; `getAuthorities()` maps the single `Role` enum value to one `SimpleGrantedAuthority`. Uses a nested static `Builder` rather than a fluent-setter or Lombok-generated builder.

Role `enum`

`ROLE_USER` and `ROLE_ADMIN`. New accounts are always assigned `ROLE_USER` (see `AuthService.register`).

Design Observation: `ROLE_ADMIN` is declared but is not verified anywhere in the codebase.

`@EnableMethodSecurity` is enabled in `SecurityConfig`, yet no controller or service uses `@PreAuthorize`, `hasRole`, or `hasAuthority`. Consequently, every authenticated user is granted identical access to all task endpoints regardless of assigned role, and no administrator-only endpoint exists anywhere in the codebase.

UserRepository / UserController

`UserRepository` adds `findByEmail` and `existsByEmail` to the standard `JpaRepository`. `UserController` exposes a single endpoint, `GET /api/users/me`, which resolves the caller directly from `@AuthenticationPrincipal`; no explicit lookup is required, as the JWT filter has already placed the full `User` entity within the security context.

Package: `task`

Task `@Entity`

Fields: `title`, `description`, `priority`, `status`, `dueDate`, a `@ManyToOne` lazy-fetched `user` (the owner, non-nullable), plus auditing timestamps. Indexed on `user_id`, `status`, `priority`, and `due_date` — exactly the columns the search/filter feature queries against.

TaskRepository

Extends both `JpaRepository<Task, Long>` and `JpaSpecificationExecutor<Task>`. The `JpaSpecificationExecutor` is what allows `TaskService` to compose dynamic, type-safe `WHERE` clauses at runtime. Also declares `findByIdAndUserId` — the single method that enforces per-user ownership at the database query level for every read/update/delete.

TaskSpecifications

A static utility class producing reusable `Specification<Task>` predicates: `ownedBy(userId)`, `matchesKeyword(keyword)` (case-insensitive `LIKE` over title OR description), `hasStatus(status)`, `hasPriority(priority)`. These are combined with `.and()` in the service, so the final SQL `WHERE` clause only ever contains the filters actually supplied by the caller.

TaskService

The central business logic of the task feature. `search()` builds the specification chain and a safety-checked `Pageable` (sort field whitelisted against a hardcoded `SORTABLE_FIELDS` set to prevent property-path injection; page size clamped to a range of 1–100). `getOwnedTask()` is the single control point through which every read, update, and delete operation passes, throwing `ResourceNotFoundException` if the task does not belong to the calling user. This means a 404 response (rather than 403) is returned for another user's task, so that the existence of the task is not disclosed either. `create()`, `update()`, and `delete()` are all `@Transactional`.

TaskController

Exposes `GET/POST /api/tasks` and `GET/PUT/DELETE /api/tasks/{id}`. Every method takes `@AuthenticationPrincipal User user` as its first parameter; the authenticated owner is always implicit and is never taken from the request body or path, which prevents any possibility of operating on another user's data through a forged request payload.

DTOs — `TaskRequest`, `TaskResponse`, `PageResponse<T>`

`TaskRequest` validates `title` (required, ≤ 150 chars), `description` (≤ 5000 chars), and requires non-null `priority` / `status` enums. `PageResponse<T>` is a framework-agnostic pagination envelope (`content` , `page` , `size` , `totalElements` , `totalPages` , `first` , `last`) that wraps Spring Data's `Page<E>` without leaking Spring types into the JSON contract.

Package: `exception`

GlobalExceptionHandler `@RestControllerAdvice`

Six handlers, each mapping to one HTTP status: `MethodArgumentNotValidException` $\rightarrow$ 400 (with a per-field error list), `MethodArgumentTypeMismatchException` $\rightarrow$ 400, `ResourceNotFoundException` $\rightarrow$ 404, `EmailAlreadyExistsException` $\rightarrow$ 409, `BadCredentialsException` $\rightarrow$ 401, `AccessDeniedException` $\rightarrow$ 403, and a catch-all `Exception` $\rightarrow$ 500 with a generic message (so internal error details are never leaked to the client).

ApiError / FieldValidationError

The single record shape every error response conforms to: `timestamp` , `status` , `error` , `message` , `path` , and an optional `fieldErrors` list (only present for validation failures, thanks to `@JsonInclude(NON_NULL)`).

Frontend classes

AuthService

Manages the client-side session. `login()` and `register()` issue POST requests to the backend and, via an RxJS `tap` , persist `{token, user}` to `localStorage` . It exposes a signal `user` and a derived `isAuthenticated` computed signal, to which the navbar, guards, and root component all react. `logout()` clears storage and navigates to `/login` .

TaskService

A lightweight `HttpClient` wrapper for `/api/tasks` . It builds `HttpParams` for search, filter, sort, and pagination, and exposes `list` , `get` , `create` , `update` , and `delete` as observables. It maintains no internal state.

authGuard / guestGuard

Functional `CanActivateFn` implementations. `authGuard` redirects unauthenticated users away from `/dashboard` to `/login` ; `guestGuard` applies the inverse rule to `/login` and `/register` , keeping signed-in users on `/dashboard` .

authInterceptor / errorInterceptor

`authInterceptor` attaches `Authorization: Bearer <token>` to every outgoing request when a token is present. `errorInterceptor` centralizes failure handling: on a 401 response from any endpoint other than `/auth/` , it logs the user out; for every error, it normalizes the backend's `ApiError` body into a plain `Error` with a human-readable message that the interface can display directly.

DashboardComponent

TaskFormComponent

The primary authenticated screen. It holds all list state as signals (`tasks` , `loading` , `error` , `page` , `search` , `statusFilter` , `priorityFilter` , `sortValue`) and calls `TaskService.list()` on every change (debounced by 300 milliseconds for free-text search). It opens `TaskFormComponent` as a modal for creation and editing, and requires confirmation before deletion.

The create/edit modal, implemented using Angular's `input()` / `output()` signal APIs rather than the `@Input()` / `@Output()` decorators. It pre-fills from the `task` input when editing an existing task and emits a validated `TaskRequest` via the `saved` output. It closes on the Escape key (`@HostListener`) or on a backdrop click.

How the Frontend, Backend, and Database Communicate

All communication follows a synchronous request/response HTTP and JSON model; no WebSockets, Server-Sent Events, or polling mechanisms exist anywhere in the codebase.

1 **Angular component** → AuthService / TaskService

A component (e.g. `DashboardComponent`) calls a method on an injected service, which builds an `HttpClient` call (GET/POST/PUT/DELETE) against `environment.apiUrl` (`http://localhost:8080/api`).

2 **authInterceptor**

Attaches `Authorization: Bearer <token>` from `localStorage` (via `AuthService.getToken()`) if a token is present.

3 **Browser** → network (CORS)

Since the SPA (4200) and API (8080) are different origins, the browser applies CORS.

`SecurityConfig.corsConfigurationSource()` explicitly allows the configured origin(s), methods GET/POST/PUT/PATCH/DELETE/OPTIONS, all headers, and exposes the `Authorization` header back to the client.

4 **Spring servlet filter chain** → JwtAuthenticationFilter

Runs before any controller. Validates the bearer token and — if valid — populates

`SecurityContextHolder` with the authenticated `User`.

5 **Controller** → Service → Repository

The matched `@RestController` method delegates to a `@Service`, which uses a Spring Data repository to talk to the database via Hibernate-generated SQL over JDBC.

6 **PostgreSQL** → Hibernate → entities

Result rows are hydrated into JPA entities (`User`, `Task`), mapped to response DTOs, and serialized to JSON by Jackson inside the `ResponseEntity`.

7 **errorInterceptor** ← HTTP response

On success, the JSON flows straight back into the calling service's `Observable`. On failure, this interceptor first normalizes the backend's `ApiError` body into a friendly `Error` (and forces logout on an unexpected 401) before it reaches the component.

8 **Component** → signals → template

The component's `subscribe({ next, error })` handler updates its signals; Angular's change detection re-renders the template reactively.

Authentication Flow (JWT)

Authentication is **stateless**: `SecurityConfig` sets `SessionCreationPolicy.STATELESS`, so no server-side session or cookie is ever created. Identity is carried entirely within a signed JWT that the client stores and resends on every request. No refresh-token mechanism exists within this codebase; the token remains valid for a fixed lifetime (24 hours by default), and once expired, the user is required to sign in again.

7.1 Registration

1 RegisterComponent

Submits `{ name, email, password }` to `POST /api/auth/register` — permitted without a token (`SecurityConfig` whitelists `/api/auth/**`).

2 AuthService.register()

`userRepository.existsByEmail()` — if the email is taken, throws `EmailAlreadyExistsException` (→ **409 Conflict** via `GlobalExceptionHandler`).

3 Password hashing

The plaintext password is hashed with `BCryptPasswordEncoder` before a `User` (role `ROLE_USER`) is built and saved via `UserRepository`.

4 JwtService.generateToken()

Issues a JWT: subject = email, `issuedAt` = now, `expiration` = now + `app.jwt.expiration-ms`, signed with the HMAC key derived from `app.jwt.secret`.

5 Response → AuthService.storeSession()

`AuthResponse{token, "Bearer", user}` returns with **201 Created**. The frontend persists token + user to `localStorage`, sets the `user` signal, and the router navigates to `/dashboard`.

7.2 Login

1 LoginComponent

Submits `{ email, password }` to `POST /api/auth/login`.

2 AuthService.login()

Builds a `UsernamePasswordAuthenticationToken` and calls `AuthenticationManager.authenticate()`.

3 DaoAuthenticationProvider → AppUserDetailsService

Loads the `User` by email via `UserRepository.findByEmail`, then compares the submitted password against the stored BCrypt hash using the same `PasswordEncoder` bean.

✗ On mismatch

Spring Security throws `BadCredentialsException` → `GlobalExceptionHandler` → **401** `{"message": "Invalid email or password"}`.

4 On success → JwtService.generateToken()

A new JWT is issued for the authenticated principal and returned as an `AuthResponse`; the frontend stores the session exactly as in registration.

7.3 Authenticated Request (Example: Dashboard Load)

1 authInterceptor

Reads the stored token and adds `Authorization: Bearer <token>` to the outgoing request.

2 JwtAuthenticationFilter

Extracts the token, calls `jwtService.extractUsername()`, loads `UserDetails` via `AppUserDetailsService`, then `isTokenValid()` checks subject match + non-expiry.

3 SecurityContextHolder

If valid, a `UsernamePasswordAuthenticationToken` carrying the full `User` entity (not just a username string) is set as the current `Authentication` — this is exactly what later lets `@AuthenticationPrincipal User user` resolve directly in controllers.

✗ Missing / invalid / expired token

The filter never throws — it silently leaves the request unauthenticated (or clears the context on a parse error). `SecurityConfig`'s `.anyRequest().authenticated()` then rejects the request, and `JwtAuthenticationEntryPoint` writes a uniform **401** `ApiError` JSON body.

4 errorInterceptor (frontend)

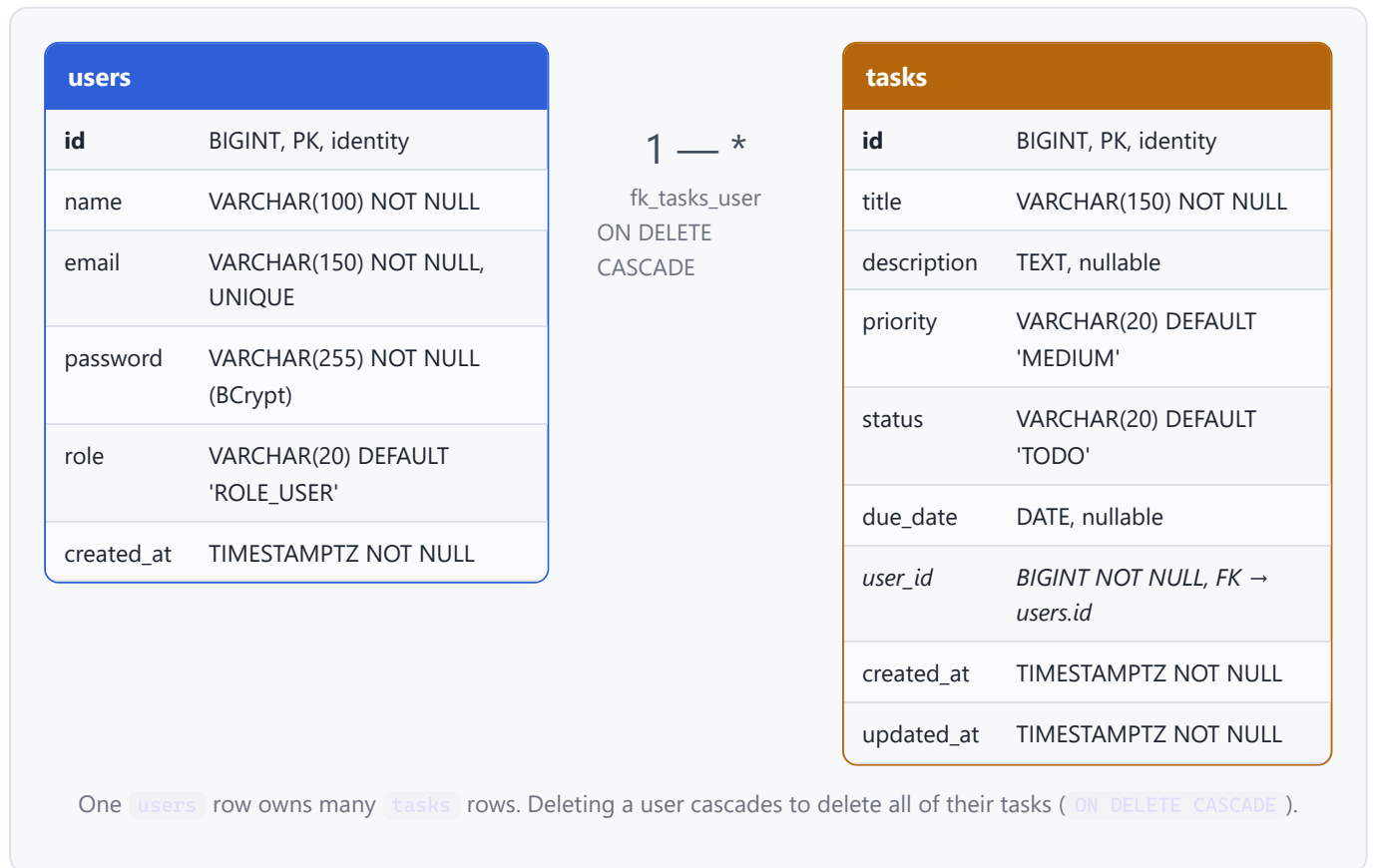
On a 401 response from any endpoint other than `/auth/`, calls `auth.logout()`, clearing storage and redirecting to `/login`, so that an expired token results in a clean re-authentication prompt rather than a persistent error state.

Known Limitation: `Role.ROLE_ADMIN` exists within the data model but is not enforced anywhere; every authenticated user holds identical permissions. No logout endpoint or token blacklist exists on the backend; logout is implemented purely as a client-side action (clearing `localStorage`). Because tokens are stateless and self-contained, a token remains cryptographically valid until its natural expiry, even following a client-side logout.

SECTION 8

Database Structure

The schema comprises exactly two tables, defined by the `User` and `Task` `@Entity` classes and mirrored in `database/schema.sql`. In development, Hibernate creates and updates these tables automatically (`ddl-auto: update`); the SQL file constitutes the equivalent hand-written DDL intended for use with `ddl-auto: validate` in production.



Indexes

Index	Column	Purpose
uk_users_email	users.email	Enforces one account per email; backs <code>existsByEmail</code> / <code>findByEmail</code> lookups
idx_tasks_user	tasks.user_id	Every task query is scoped by owner (<code>TaskSpecifications.ownedBy</code>); this index supports the primary access path for every request
idx_tasks_status	tasks.status	Backs the status filter
idx_tasks_priority	tasks.priority	Backs the priority filter
idx_tasks_due_date	tasks.due_date	Backs sorting/filtering by due date

Entity-Relational Mapping Notes

- `User` and `Task` both use `GenerationType.IDENTITY` primary keys, corresponding to PostgreSQL's `GENERATED BY DEFAULT AS IDENTITY`.
- `Task.user` is declared as `@ManyToOne(fetch = FetchType.LAZY, optional = false)`; the owner is never optional, and the association is not eagerly joined unless explicitly navigated.

- `priority`, `status`, and `role` are persisted as their enum **name** (`@Enumerated(EnumType.STRING)`), such that stored values are human-readable strings (for example, `"HIGH"`, `"TODO"`) rather than ordinals.
- `open-in-view: false` is set in `application.yml`, which avoids the lazy-loading-in-view anti-pattern; all entity access occurs within the `@Transactional` service methods, prior to DTO projection.

API Request Flow

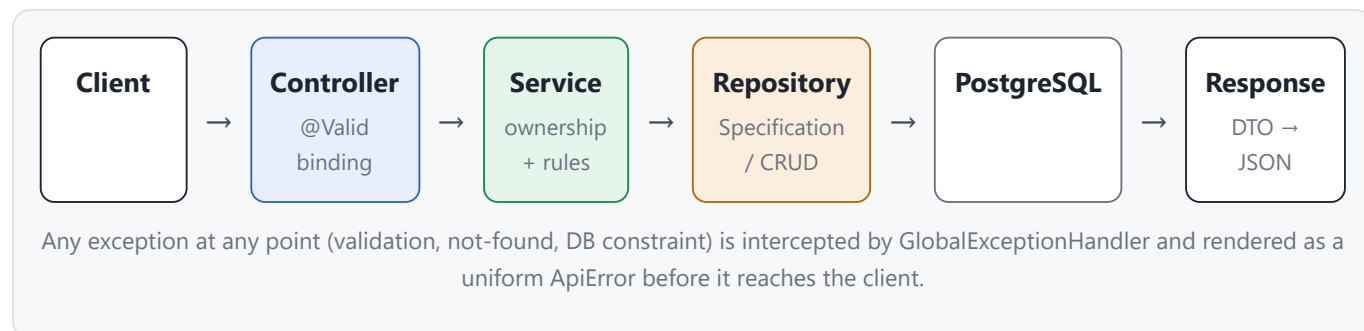
Client → Controller → Service → Repository → Database → Response, traced through a concrete example:

```
GET /api/tasks?search=report&status=TODO&priority=HIGH&sortBy=dueDate&direction=asc&page=0&size=10
```

- 1 **DashboardComponent** → `TaskService.list()`
Builds an `HttpParams` object from the current search/filter/sort/page signals and issues an `HttpClient.get`.
- 2 **authInterceptor**
Adds the bearer token header.
- 3 **JwtAuthenticationFilter**
Authenticates the request and sets `SecurityContextHolder` before the request reaches `DispatcherServlet` routing.
- 4 **TaskController.list()**
Resolves `@AuthenticationPrincipal User user` and binds query parameters (`search` , `status` , `priority` , `page` , `size` , `sortBy` , `direction` , with controller-level defaults of `page=0` , `size=10` , `sortBy=createdAt` , `direction=desc` when the frontend does not specify them). Calls `taskService.search(...)`.
- 5 **TaskService.search()**
Begins from `TaskSpecifications.ownedBy(user.getId())` , which is always applied regardless of the parameters supplied by the caller, then conditionally combines `matchesKeyword` , `hasStatus` , and `hasPriority` only for the parameters actually provided. Builds a validated `Pageable` : `sortBy` is checked against the `SORTABLE_FIELDS` whitelist (defaulting to `createdAt` if not recognized), and `size` is clamped to a range of 1–100.
- 6 **TaskRepository** (`JpaSpecificationExecutor`)
Hibernate translates the composed `Specification` + `Pageable` into one parameterized `SELECT` (with a `WHERE user_id = ? AND (lower(title) LIKE ? OR lower(description) LIKE ?) AND status = ? AND priority = ? ORDER BY due_date ASC LIMIT ? OFFSET ?`) plus a matching `COUNT` query for `totalElements` , executed against **PostgreSQL** over JDBC.
- 7 **Result** → `Page<Task>` → `PageResponse<TaskResponse>`
Back in the controller, `PageResponse.from(result, TaskResponse::from)` projects each `Task` entity into a `TaskResponse` DTO and wraps the page metadata into the framework-agnostic envelope.
- 8 **Response**
`ResponseEntity.ok(pageResponse)` is serialized to JSON by Jackson and sent back over HTTP.
- 9 **Angular** → signals → template
`DashboardComponent` 's subscription updates `tasks` , `totalElements` , and `totalPages` signals; the grid re-renders.

Write Operations (Create, Update, Delete)

The same layering applies to mutating operations, with one addition: **Bean Validation** executes before the controller method body runs. `@Valid @RequestBody TaskRequest request` on `create` and `update` triggers `MethodArgumentNotValidException` on any constraint violation (blank title, oversized description, null priority or status), which is caught by `GlobalExceptionHandler` and returned as **400** with a per-field `fieldErrors` array before `TaskService` is invoked at all. For `update` and `delete`, `TaskService.getOwnedTask()` serves as the ownership gate: it re-fetches the record by `(id, user.getId())`, so that attempting to modify another user's task returns **404 Not Found**, identical to the response for a genuinely nonexistent identifier, which avoids revealing whether the identifier belongs to another user.



API Reference

Base path `/api`. Every route except `/api/auth/**` and `/actuator/health` requires `Authorization: Bearer <token>`.

Method	Endpoint	Auth	Body	Description
POST	<code>/api/auth/register</code>	Public	<code>{name, email, password}</code>	Create account, returns JWT + user
POST	<code>/api/auth/login</code>	Public	<code>{email, password}</code>	Sign in, returns JWT + user
GET	<code>/api/users/me</code>	Bearer	—	Current authenticated user
GET	<code>/api/tasks</code>	Bearer	query params	List own tasks, paged & filterable
GET	<code>/api/tasks/{id}</code>	Bearer	—	Get one owned task (404 if not owner)
POST	<code>/api/tasks</code>	Bearer	<code>TaskRequest</code>	Create a task for the caller
PUT	<code>/api/tasks/{id}</code>	Bearer	<code>TaskRequest</code>	Full update of an owned task
DELETE	<code>/api/tasks/{id}</code>	Bearer	—	Delete an owned task (204)

`GET /api/tasks` query parameters

Param	Type	Default	Notes
search	string	—	Case-insensitive match on title OR description
status	TODO IN_PROGRESS DONE	—	Exact match
priority	LOW MEDIUM HIGH	—	Exact match
page	int	0	Clamped to ≥ 0
size	int	10 (backend) / 9 (frontend default)	Clamped to 1–100
sortBy	string	createdAt	Whitelisted: createdAt, updatedAt, dueDate, priority, status, title
direction	asc desc	desc	Case-insensitive

`TaskRequest` shape

```
{
  "title": "Write project README",
  "description": "Cover setup + API",
  "priority": "HIGH",
  "status": "IN_PROGRESS",
  "dueDate": "2026-07-15"
}
```

Error shape (`ApiError`) — returned for every non-2xx response

```
{
  "timestamp": "2026-07-15T10:22:31.442Z",
  "status": 400,
  "error": "Validation Failed",
  "message": "One or more fields are invalid",
  "path": "/api/tasks",
  "fieldErrors": [
    { "field": "title", "message": "Title is required" }
  ]
}
```

This document was compiled through direct inspection of the source code in `task-management-system/backend`, `/frontend`, and `/database`. No functionality has been assumed or inferred beyond what is implemented in the code.